

한국식 4구 당구 RL 프로젝트 — 처음부터 끝까지

한국식 4구 당구 RL 프로젝트 — 처음부터 끝까지

이 문서를 읽는 데 사전 지식이 필요하지 않다. 강화학습이 뭔지, 당구가 뭔지, 신경망이 뭔지 몰라도 된다. 모르는 단어가 나오면 그 자리에서 정의된다.

읽는 시간: 천천히 읽으면 1시간. 스킵하면서 읽으면 20분.

작성 시점: 2026-04-29.

목차

이 문서는 **순서대로 읽어야** 이해된다. 각 섹션은 이전 섹션에서 정의된 개념만 사용한다.

Part 1 — 무슨 프로젝트인가 1. 한 줄 요약 2. 강화학습이 뭐야 — 처음부터 3. 한국식 4구 당구가 뭐야 4. 왜 4구를 강화학습으로 푸냐

Part 2 — 우리가 만든 것 5. 큰 그림 — 우리 시스템 4단계 6. 1단계: 시뮬레이터 만들기 7. 2단계: 강화학습이 쓸 수 있게 감싸기 (Gym 환경) 8. 3단계: 환경 변형용 Wrapper 9. 4단계: RL 알고리즘으로 학습 (SAC)

Part 3 — 부속 인프라와 결과 10. 실험 자동화 인프라 11. 분석, 시각화, 페이퍼 12. 지금까지 한 일 (Phase A → I) 13. 현재 결과 — Plain SAC 66.7%

Part 4 — 앞으로 14. Phase II 로드맵 — Solver 80%+ 15. 페이퍼 방향성

Part 5 — RLHF 확장 (별도) 16. RLHF가 뭐야 — 처음부터 17. 4구에 RLHF가 왜 필요한가 18. 우리가 이미 시도한 RLHF — Phase F, G 19. 0% 벽의 정체 — Reward Hacking 20. 0%를 깨는 처방 — Phase J, K, L

부록 21. 코드 / 파일 위치 22. 용어집 23. 참고 문헌 24. FAQ 25. 신규 팀원 가이드

Part 1 — 무슨 프로젝트인가

1. 한 줄 요약

“한국식 4구 당구를 컴퓨터가 잘 치게 만들고, 그 과정에서 강화학습 알고리즘이 어디까지 풀 수 있는지 본다.”

이 한 줄에 들어있는 단어들 — “한국식 4구”, “강화학습” — 을 차근차근 풀어 설명한다. 그 다음에 “그래서 뭘 만들었나”를 보여준다.

프로젝트 동기 (솔직히): KAIST CS377 강화학습 수업의 1개월 프로젝트. 동시에 진학하고 싶은 두 교수님 랩 — 이기민(KAIST AI 대학원), 임재환(CLVR/USC→KAIST) — 어필용.

2. 강화학습이 뭐야 — 처음부터

강화학습(Reinforcement Learning, **RL**)을 한 번도 안 본 사람을 위해.

2.1. 비유 — 강아지 훈련

강아지에게 “앉아”를 가르친다고 하자.

- 강아지가 우연히 엉덩이를 바닥에 댄다 → 너가 간식을 준다.
- 강아지가 엉덩이를 안 댄다 → 간식 없음.
- 이걸 100번 반복한다.
- 강아지는 점점 “앉아”라는 말을 들으면 엉덩이를 대는 행동을 더 자주 한다.

이게 **강화학습의 본질**이다. 보상(간식)을 받은 행동은 더 자주 하고, 못 받은 행동은 덜 한다.

컴퓨터 버전: - **AI** = 강아지 - **환경** = 너 (간식 주는 사람) + 거실 (강아지가 활동하는 공간) - **보상** = 간식 - **행동 (action)** = 강아지가 할 수 있는 동작들 (앉기, 짖기, 누워있기, 등) - **상태(state)** = 강아지가 보는 거실 모습 + “앉아”라는 명령이 떨어진 상황 - **정책(policy)** = “이 상황(state)에서 이 행동(action)을 한다”라는 강아지의 머릿속 규칙

2.2. RL의 핵심 함수 — 정책

AI가 학습하는 건 결국 **정책 함수** 하나다:

정책 = “지금 보이는 상태(state)를 보면, 어떤 행동(action)을 해야 보상이 최대로 나올까?” 라는 결정 규칙

기호로 $\pi(\text{action} | \text{state})$ 이라고 쓴다. 신경망(컴퓨터의 뇌 같은 거)으로 이 정책을 표현한다.

2.3. 학습 사이클

학습은 다음 사이클을 수만 번 반복: 1. AI가 현재 state를 본다 2. 정책으로 action을 결정한다 3. 환경에서 그 action을 실행한다 4. 환경이 “다음 state + 받은 reward”를 돌려준다 5. AI는 “이 state에서 이 action이 좋았다 / 나빴다”를 학습한다 → 정책이 조금 좋아진다

수만 번 돌리면 정책이 점점 좋아진다.

2.4. 우리 4구로 매핑하면

RL 개념	4구에서는
AI / 정책	4구 치는 컴퓨터 선수
환경	당구대 (시뮬레이터)
state (상태)	공 4개의 위치, 속도, 회전
action (행동)	큐를 어떻게 칠지 (각도, 세기, 어디 칠지)
reward (보상)	점수 났으면 +1, 아니면 0
episode (한 게임 세션)	한 샷 또는 한 inning

이 매핑이 **이 프로젝트의 출발점**. 여기서 모든 게 따라온다.

3. 한국식 4구 당구가 뭐야

당구를 한 번도 안 친 사람을 위해.

3.1. 당구의 두 종류 — 포켓과 카롬

당구는 크게 두 종류:

포켓 게임 (미국식 풀, 영국식 스누커): - 당구대 6개 모서리에 **구멍(포켓)** 이 있다 - 공을 구멍에 떨어뜨리는 게 목표

카롬 게임 (3쿠션, 4구): - **포켓이 없다** (구멍이 없음) - 공이 절대 떨어지지 않는다 - 공을 다른 공에 부딪히게 만드는 게 점수

한국식 4구는 **카롬 게임의 변형**이다.

3.2. 4구 룰

테이블 위에 공이 4개: - 흰공 1개 (내가 칠 큐볼) - 흰공 1개 (상대 큐볼, 단순히 장애물 역할) - 빨간공 2개

내가 큐스틱으로 내 흰공을 친다. 그 흰공이 굴러다니면서: - **양쪽 빨간공을 둘 다 맞춌 + 상대 흰공은 안 맞춌 → +1점**
- 빨간공 하나만 맞춌 → 0점 - 상대 흰공 맞춌 → **파울** (0점 + 다음 차례는 상대)

수식으로 쓰면:

$$\text{score} = \begin{cases} +1 & \text{if } (\text{빨간공}_1 \wedge \text{빨간공}_2) \wedge \neg \text{상대공} \\ 0 & \text{otherwise} \end{cases}$$

3.3. Inning(이닝) 룰

한 번 점수 내면 **계속 친다**. 못 맞추거나 파울 나면 그 회 끝. 이게 한 **inning**(이닝).

샷 1 → 점수 +1 → 계속
샷 2 → 점수 +1 → 계속
샷 3 → 점수 0 → inning 끝, 누적 점수 = 2

진짜 게임은 누적 점수로 승부 보는데, 우리 학습은 일단 **한 샷에 1점 내는 것부터** 목표로 함 (3.5에서 자세히).

3.4. 큐스틱으로 공 치기 — 4가지 자유도

내가 큐로 공을 칠 때 결정할 수 있는 4가지:

1. **방향** (θ): 어느 방향으로 칠까. $0^\circ \sim 360^\circ$.
2. **세기** (power): 얼마나 세게 칠까. 0(살살) $\sim$ 1(최대).
3. **좌우 빗맞춤** (a): 공의 좌우 어디를 칠까. -1(왼쪽) $\sim$ +1(오른쪽).
4. **상하 빗맞춤** (b): 공의 위아래 어디를 칠까. -1(아래/백스핀) $\sim$ +1(위/탑스핀).

이 4개가 우리 RL의 **action**이다. 그래서 4-dim continuous action.

3.5. 학습 단순화 — “한 샷 = 한 episode”

진짜 4구는 inning(여러 샷)으로 점수 누적되지만, 우리 RL은 일단 “한 샷에 점수 낼 수 있는가” 부터 학습. 이게 단순해서 분석/디버깅 쉬움. 나중에 inning으로 확장.

4. 왜 4구를 강화학습으로 푸나

여러 게임 중 4구를 고른 이유.

4.1. 우리한테 좋은 점들

1. **룰이 한 줄로 정의됨** — `score = 1 if (양쪽 빨간공  $\wedge$   $\neg$ 상대공) else 0`. 이거 RL 입장에서 reward 함수가 깔끔해서 좋음.
2. **공이 사라지지 않음** — 포켓이 없으니 공 4개가 항상 테이블 위. AI가 보는 state 차원이 고정 (28-dim, 자세한 건 7장에서). 포켓 게임은 공이 떨어지면서 차원이 가변이라 RL이 더 어려움.
3. **시물이 빠름** — 한 샷 5~10밀리초 (직접 numpy로 짜면). 5만 샷 학습이 5분.
4. **연속 행동** — (θ , power, a , b) 4개의 실수. SAC 같은 표준 RL 알고리즘이 잘 다루는 영역.
5. **자연스러운 대칭** — 좌우 뒤집어도, 위아래 뒤집어도, 큐볼끼리 swap해도 4구는 같은 게임. 데이터 증강에 활용 가능.

4.2. 학술적 가치

1. **영문 RL 논문에 4구는 거의 없음** — novelty 확보.
2. **한국 도메인** — 한국 교수님 어필 스토리.
3. **Sparse-binary reward** — 보상이 0 아니면 1, 그것도 가끔만 나옴. 이게 RL의 어려운 영역이라 알고리즘 분석에 좋음.

4.3. 다른 후보는 왜 탈락했나

후보	탈락 이유
MicroRTS / OpenSpiel	wargame RL 페이퍼 너무 많음, novelty 약함
MuJoCo / Adroit	표준 환경이지만 도메인 흥미 약함
Atari	이산 행동, 우리 hand이 익히지 않음

Part 2 — 우리가 만든 것

5. 큰 그림 — 우리 시스템 4단계

여기서부터는 우리가 만든 시스템을 차례차례 본다. 큰 그림은 4단계:

```

[4단계] AI (SAC 알고리즘)을 학습시킨다
  ↑
[3단계] 환경을 변형해서 다양한 실험 셋업을 만든다 (Wrapper)
  ↑
[2단계] 시뮬레이터를 RL이 사용할 수 있는 인터페이스로 감싼다 (Gym 환경)
  ↑
[1단계] 컴퓨터 안에 4구 당구대를 만든다 (시뮬레이터)

```

각 단계는 위에 있는 단계의 토대가 된다. 1단계 없으면 2단계 못 만들고, 2단계 없으면 3단계 못 만든다.

이제 1단계부터 차례로.

6. 1단계: 시뮬레이터 만들기

6.1. 왜 시뮬레이터부터 만드나

진짜 당구대를 못 쓴다. AI가 학습하려면 **5만 번 샷**을 쳐봐야 하는데: - 진짜 당구대로 1샷 30초 잡아도 5만 샷 = **400시간** = 17일 (불가능) - 컴퓨터 시뮬은 1샷 5~10ms = 5만 샷 = 5~10분 (가능)

그래서 **컴퓨터 안에 가짜 당구대**를 만든다. 그게 시뮬레이터.

6.2. 시뮬레이터가 하는 일

딱 한 가지: > “**공 4개의 현재 위치/속도/회전을 받아서, 다음 순간 어떻게 변할지 계산**”

이걸 0.001초 단위로 반복해서 공이 다 멈출 때까지 굴린다. 그러면 한 샷 끝.

이걸 정확히 하려면 **당구의 4가지 물리 사건**을 다 다뤄야 한다:

1. 큐로 흰공을 처음 칠 때 → 흰공이 어떤 속도와 회전을 갖나
2. 공이 천 위에서 굴러갈 때 → 어떻게 감속하나

3. 두 공이 부딪힐 때 → 어떻게 튕기나
4. 공이 쿠션(벽)에 부딪힐 때 → 어떻게 튕기나

각각의 물리는 학계에 정립된 표준 수식이 있다. 우리는 그걸 **numpy**로 직접 짤다.

6.3. 왜 PoolTool 라이브러리 안 쓰고 자작했나

PoolTool(github.com/ekiefl/pooltool)은 사람이 보고 즐기는 당구 시뮬 게임. 무료 오픈소스라 우리가 그냥 쓸 수도 있다. 그런데 안 썼다.

이유: PoolTool이 너무 많은 일을 한다.

PoolTool에 있는 기능	4구 학습에 필요?
포켓 (구멍) 처리	✗ 4구는 구멍 없음
점프샷, 마세 (큐를 세워치기)	✗
천 결, 온도, 습도 효과	✗
속도 의존 쿠션 반발계수	✗
3D 그래픽, UI, 사운드	✗

왜 이게 문제냐: 한 샷 칠 때마다 PoolTool은 위 기능들을 “혹시 필요할까?” 하고 다 거쳐 간다. Python에서 함수 호출 한 번이 약간 느려서 **한 샷에 ~100ms** 걸린다. 5만 샷 학습 = **1.5시간**. 자작 시뮬은 5~10ms → 5분. **약 10~20배 차이.**

학부 1개월 프로젝트에서 이 차이는 결정적이다.

해결: PoolTool에서 우리에게 필요한 4개 모듈만 골라 numpy로 다시 짤다. 코드 출처는 `physics/README.md` 에 다 명시.

6.4. 4가지 물리 자세히

이제 시뮬레이터의 4가지 컴포넌트를 하나씩.

6.4.1. 큐로 흰공 치기 (Cue Impact) — `cue_impact.py`

무엇을 모델링하나: 큐스틱이 멈춰있는 흰공의 한 점을 째는 순간 친다. 그 결과로 흰공이 (속도, 회전)을 갖게 된다.

적용한 식 (Marlow 1995):

$$v_{\text{eff}} = \frac{2 V_0}{1 + \frac{m}{M} + \frac{5}{2}(a^2 + b^2)}$$

각 기호 풀이:

기호	뜻	일상 비유
V_0	큐스틱 끝의 속도 (m/s)	“얼마나 세게 휘둘렀나”
m	공 무게 (0.21 kg)	“맞는 놈”
M	큐 무게 (0.55 kg)	“때리는 놈”
a	좌우 빗맞춤 ($-1 \sim +1$)	“공 옆구리를 쳤나”
b	위아래 빗맞춤 ($-1 \sim +1$)	“공 머리/턱을 쳤나”
v_{eff}	결과 공 속도	“공이 얼마나 빠르게 나가나”

해석 — 빗맞춤수록 속도가 줄어든다:

- 정중앙을 침 ($a = b = 0$) $\rightarrow v_{\text{eff}} \approx 1.45 V_0$ (큐보다도 빠름)
- 가장자리를 침 ($a^2 + b^2 = 1$) $\rightarrow v_{\text{eff}} \approx 0.51 V_0$ (절반 이하)

왜 그런가: 빗맞으면 에너지의 일부가 “공을 회전시키는 데” 빨려간다. 그래서 직진력은 줄고 회전이 생긴다.

비유: 축구공을 정중앙으로 차면 멀리 가지만, 옆구리를 차면 빙글빙글 돌면서 멀리 안 간다.

부산물 — 회전 발생:

$$\omega_z = -\frac{5 v_{\text{eff}}}{2R} \cdot a \quad (\text{좌우 회전, 즉 사이드 스핀})$$

$$\omega_{\perp} = +\frac{5 v_{\text{eff}}}{2R} \cdot b \quad (\text{앞뒤 회전, 즉 탑/백 스핀})$$

여기서 R 은 공 반지름. 공의 어디를 쳤느냐가 어떤 회전이 생기느냐를 결정한다:

- 공의 위쪽을 침 ($b > 0$) $\rightarrow$ **탑스핀** (밀어치기). 맞고 나서 더 굴러감.
- 공의 아래쪽을 침 ($b < 0$) $\rightarrow$ **백스핀** (끌어치기). 맞고 나서 다시 돌아옴.
- 공의 오른쪽을 침 ($a > 0$) $\rightarrow$ **사이드 스핀** (잉글리쉬). 쿠션 맞을 때 진로가 휘어짐.

이 식들이 **우리 RL의 action** (θ , power, a , b)의 **물리적 의미**를 정의한다. AI가 학습하는 게 결국 “어떤 (θ , power, a , b) 조합이 점수로 이어지나”를 배우는 일이다.

6.4.2. 공이 천 위에서 굴러가기 (Free Flight) — `dynamics.py`

큐가 친 흰공은 처음에 **속도와 회전이 안 맞아 있다**. 즉 “공이 앞으로 빠르게 가는데 회전은 별로 없는” 상태. 이때 공이 천에 닿아 있으니까 **두 단계**를 거친다:

단계 1 — 슬립(slip, 미끄러짐):

공이 굴러야 할 회전보다 빠르게 가니까 천에 비벼진다. 마찰이 큼: - 미끄럼 마찰계수 $\mu_s = 0.20$ - 미끄럼 속도 $\mathbf{u}$ (공이 천 위에서 비벼지는 속도)는 다음 식으로 줄어든다:

$$\frac{d\mathbf{u}}{dt} = -\frac{7}{2}\mu_s g \hat{\mathbf{u}}$$

여기서 g 는 중력가속도. 비벼지는 속도가 0이 되는 시간:

$$t_{\text{roll}} = \frac{|\mathbf{u}|}{\frac{7}{2}\mu_s g}$$

계수 $\frac{7}{2}$ 가 어디서 나왔냐면 — 공의 회전 관성 $I = \frac{2}{5}mR^2$ + 직선 운동을 합친 표준 결과다 (학부 일반물리에 나온다).

비유: 볼링공을 던지면 처음엔 미끄러져 가다가 점차 회전이 붙는다. 그 “미끄러지는 시간”이 t_{roll} 이다.

단계 2 — 롤(roll, 순수 굴러감):

회전과 속도가 딱 맞아서 미끄럼 없이 굴러간다. 마찰이 훨씬 작음: - 구름 마찰계수 $\mu_r = 0.018$ (슬립의 약 10배 작음)

$$\frac{d\mathbf{v}}{dt} = -\mu_r g \hat{\mathbf{v}}$$

천천히 감속하다가 멈춘다.

한 가지 더 — 수직축 회전 (ω_z):

공이 그 자리에서 빙글빙글 도는 회전 (사이드 스핀의 효과). 이상화된 점접촉 모델에서 이건 **앞으로 가는 운동에 영향을 안 준다**. 그저 자기 자리에서 돌 뿐. 단, 쿠션에 닿을 때만 진로를 휘게 만든다 (다음 컴포넌트에서).

6.4.3. 공-공 충돌 — `collisions.py::resolve_ball_ball`

가장 단순.

가정: 두 공이 같은 무게, 충돌은 잠깐, 공-공 사이 마찰은 무시.

적용한 식 (등질량 + 반발계수):

충돌선(두 공 중심을 잇는 선)을 따라서만 속도를 교환:

$$v'_1 = \frac{(1-e)v_1 + (1+e)v_2}{2}$$

$$v'_2 = \frac{(1+e)v_1 + (1-e)v_2}{2}$$

- e 는 반발계수, ≈ 0.94 (1에 가까움 = 거의 탄성충돌, 약간의 에너지 손실)
- v_1, v_2 는 충돌선 방향 속도

해석: - $e = 1$ (완벽 탄성충돌)이면 두 공이 속도를 그대로 바꾼다. 멈춰있던 공이 굴러오던 공의 속도를 받고, 굴러오던 공은 멈춘다. - $e = 0.94$ 면 충돌마다 약 6% 에너지가 사라진다 (소리, 열로).

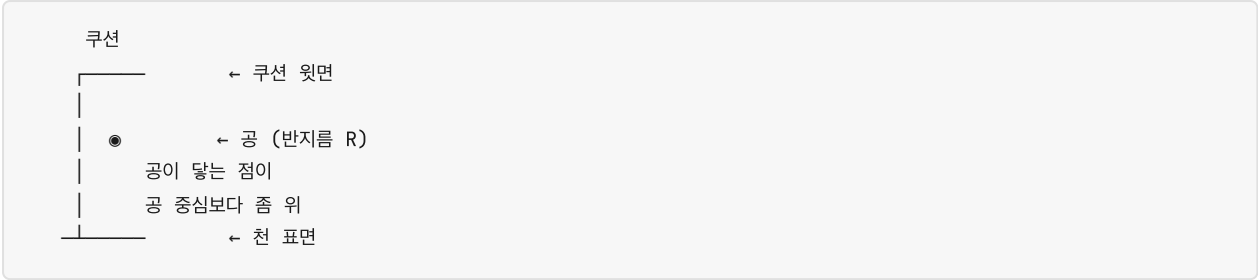
옆 방향(tangential) 속도와 회전은 **그대로 통과**한다. 진짜 당구는 회전이 다른 공에 끌림(throw) 효과가 있는데, 우리는 무시. 4구 분석에 큰 영향 없음.

6.4.4. 공-쿠션(벽) 충돌 — `collisions.py::resolve_ball_cushion`

가장 복잡. 학계 논문 한 편이 통째로 이거에 들어간다.

왜 복잡한가: 쿠션은 단순한 벽이 아니다.

당구대 쿠션의 단면을 옆에서 보면:



공이 쿠션에 닿을 때 **닿는 점이 공 중심보다 약 7.5° 위쪽**이다. 왜? 쿠션 노즈 높이 $h \approx 37\text{ mm}$ 가 공 반지름 $R \approx 32.75\text{ mm}$ 보다 살짝 크기 때문:

$$\theta_a = \arcsin\left(\frac{h}{R} - 1\right) \approx 7.5^\circ$$

왜 이 7.5°가 중요한가: 충돌이 단순히 “옆벽에서 튕기듯” 일어나면 회전이 진로에 영향을 안 준다. 근데 닿는 점이 위쪽이니까 → **사이드 스핀이 위/아래 방향 마찰을 만들고 → 쿠션 맞고 진로가 휘어든다**. 이게 카롬/4구의 핵심 기술. 비유: 농구공을 벽에 똑바로 던지면 그대로 튕긴다. 근데 던지면서 옆 회전을 주면 벽에 비벼지면서 진로가 휜다. 같은 원리.

적용한 식 (Han 2005): 한국 사람 한인환 교수가 2005년에 카롬 쿠션 충돌을 정확히 푼 논문(JMST 19권). 9개 변수 (공 속도 v_x, v_y , 회전 w_x, w_y, w_z)의 정확한 해.

너무 복잡해서 식을 풀어 적진 않는다. 우리 코드 (`collisions.py:90-126`)는 Han 논문의 식 (14), (17), (20)-(23)을 그대로 베껴 옴.

직관적 결과:

입력	결과
정면 충돌 + 회전 없음	입사각 = 반사각 (살짝 손실)
사이드 스핀 $\omega_z > 0$ (오른쪽 회전)	진로가 오른쪽으로 휘어 들어옴
강한 톱스핀	쿠션 맞고 더 빨리 굴러나옴
강한 백스핀	쿠션 맞고 속도 많이 죽음

이 네 번째 컴포넌트가 진짜 4구를 4구답게 만든다. AI가 이걸 잘 학습하면 사이드 스핀 같은 고급 기술도 발견할 수 있다.

6.5. 출처와 단순화 정리

출처 (모두 `physics/README.md` 에 정식 인용):

사용	출처
큐 임팩트 식	Marlow, <i>The Physics of Pocket Billiards</i> (1995)
천 위 운동	학부 일반물리 표준 + Marlow
공-공 충돌	일반 충돌역학
공-쿠션 충돌	Han 2005, <i>J. Mech. Sci. Tech.</i> 19:976-984
디폴트 파라미터값	PoolTool 카롬 디폴트 (Kiefl 2024 JOSS)

의도적으로 빼버린 것들 (4구 분석에 거의 영향 없거나 1개월 안에 못 해서): - 큐 기울이기 (점프샷, 마세) — 큐 각도를 0으로 고정 - 공-공 사이 회전 끌림 (throw) — 무시 - 천 결, 온도, 습도, 속도 의존 반발계수 — 다 무시

6.6. 시뮬레이터 sanity check

코드가 진짜로 물리를 맞게 푸는지 검증한 흔적들: - `artifacts/dynamics_check.png` — 슬립→를 전환, 멈추는 시간이 이론값과 일치 - `artifacts/collisions_check.png` — 사이드 스핀에 따른 쿠션 입사/반사각 측정

여기까지 정리: 1단계 끝. 한 샷 5~10ms 안에 4구 물리를 정확히 시뮬할 수 있다.

7. 2단계: 강화학습이 쓸 수 있게 감싸기 (Gym 환경)

7.1. 왜 표준 인터페이스가 필요한가

1단계에서 시뮬레이터를 만들었다. 그런데 RL 알고리즘(SAC, PPO 등)이 이 시뮬레이터를 그대로 쓸 수 있을까? 못 쓴다.

이유: RL 알고리즘은 환경마다 인터페이스가 달랐으면 매번 새로 짜야 한다. 그러면 너무 비효율적이라 학계가 표준 인터페이스를 정해놓았다.

비유: 삼성/LG/소니 TV가 각자 다른 리모컨을 쓰면 사람이 못 외운다. 그래서 “전원 / 음량 / 채널” 같은 표준 버튼을 정해놓고 모든 TV가 그걸 따른다. 그게 보편 리모컨.

RL 세계의 보편 리모컨 = **Gymnasium** (구 OpenAI Gym).

7.2. Gym이 정한 5가지 약속

Gym 환경이 되려면 다음 5가지를 제공해야 한다:

약속 1 — `observation_space` : AI가 보는 정보의 모양

“내 환경에서 AI는 28개의 숫자를 본다. 각 숫자는 $-\infty \sim +\infty$.”

우리 코드 (`env.py:78-80`):

```
self.observation_space = gym.spaces.Box(
    low=-np.inf, high=np.inf, shape=(28,), dtype=np.float32
)
```

왜 28개? 공 4개 $\times$ {x, y, vx, vy, wx, wy, wz} = 4 $\times$ 7 = 28. 즉 위치 2 + 속도 2 + 회전 3.

약속 2 — `action_space` : AI가 할 수 있는 행동의 모양

“AI는 4개 숫자를 줄 수 있다. 첫째(θ)는 $0 \sim 2\pi$, 둘째(power)는 $0 \sim 1$, 나머지(a, b)는 $-1 \sim +1$.”

우리 코드:

```
self.action_space = gym.spaces.Box(
    low=np.array([0.0, 0.0, -1.0, -1.0]),
    high=np.array([2π, 1.0, 1.0, 1.0]),
)
```

3.4절에서 본 그 4가지.

약속 3 — `reset()` : 게임을 처음 상태로 돌려라

```
def reset(self):
    self._state = TableState.initial_4ball(...) # 4구 시작 위치
    return self._obs(), info
```

4구 표준 시작 위치(canonical layout)로 공 4개를 놓고, 첫 관찰값(28개 숫자)을 돌려준다.

약속 4 — `step(action)` : 행동 한 번 받고 결과 돌려줘라

```
def step(self, action):
    cue_action = _project_action(action) # 4개 숫자를 큐 액션으로 변환
    result = simulate_shot(self._state, cue_action) # 1단계 시뮬레이션 호출
    reward = float(result["score"]) # 점수 났으면 1, 아니면 0
    return self._obs(), reward, True, False, info
```

5개를 돌려준다:

반환값	뜻	우리 4구에선
<code>obs</code>	행동 후 상태	28개 숫자
<code>reward</code>	받은 보상	0 또는 1
<code>terminated</code>	게임 룰로 끝났나?	True (한 샷=한 게임)
<code>truncated</code>	시간/횟수 제한으로 끊겼나?	False
<code>info</code>	부가 정보	event_log, foul, trajectory 등

약속 5 — `render()` : 화면 보여줘라 (선택)

우리는 browser HTML viewer로 따로 그려서 거의 빈 함수.

7.3. 이 약속만 지키면 학습이 됨

stable-baselines3(SB3)라는 라이브러리가 SAC 등 알고리즘을 이미 구현해놨다. 우리는 그냥 갖다 쓴다:

```
from stable_baselines3 import SAC
from billiards.env import Billiards4BallEnv

env = Billiards4BallEnv()
model = SAC("MlpPolicy", env)
model.learn(total_timesteps=50_000)
```

이게 끝. 3줄. SAC는 “당구”라는 걸 모른다. 그냥 “28개 숫자 받아서 4개 숫자 돌려주는 원가” 라고만 안다. Gym 약속만 맞으면 학습된다.

7.4. 우리 환경 두 가지 — 단발 + Inning

위에서 말한 환경(`Billiards4BallEnv`)은 단발: 한 step = 한 샷 = episode 끝.

근데 진짜 4구는 점수 내면 계속 친다(inning). 그래서 두 번째 환경도 만들었다:

`Billiards4BallInningEnv` (`inning_env.py`): - 점수 내면 → episode 안 끝남, 계속 step 가능 - 못 맞추거나 파울이면 → episode 끝 (`terminated = True`) - 50 샷 넘어가면 → 강제 종료 (`truncated = True`) - 한 inning 안에서 친 모든 샷의 trajectory를 stitching해서 분석/렌더 가능

7.5. 두 환경 비교가 의미 있는 이유 — Horizon

Horizon(호라이즌): episode가 얼마나 긴가, 즉 AI가 한 게임에서 몇 step 행동하는가. - 단발 환경: horizon = 1 - inning 환경: horizon = 가변 (1~50)

왜 비교가 의미 있나: AI가 단발에서 학습하면 “한 점만 어떻게든 내고 끝”으로 발달할 수 있다. inning에서 학습하면 이론적으로 “한 점 + 다음 샷 셋업”까지 고려할 수 있다. 그 차이를 보고 싶다.

우리 디자인의 핵심: 두 환경이 obs/action 모양을 완전히 동일하게 노출한다. 그래서 SAC 코드는 그대로 두고 환경만 갈아끼우면 된다:

```
# 같은 SAC 코드
model = SAC("MlpPolicy", env)
model.learn(total_timesteps=50_000)

# env 한 줄만 갈아끼움
env = Billiards4BallEnv()          # horizon=1
# 또는
env = Billiards4BallInningEnv()    # horizon=가변
```

이렇게 하면 변수 한 개(horizon)만 바뀐 채 결과를 비교할 수 있다. 다른 변수(알고리즘, obs 형태, action 형태)는 다 동일하니, 결과가 다르면 그건 horizon 때문이라고 깔끔하게 결론 내릴 수 있다. 이게 통제 실험의 기본 원리다.

7.6. 한 가지 디테일 — Action projection

SAC는 학습 초반에 이상한 값을 자주 뱉는다. 예: $\theta = 100$ (말도 안 됨), power = 5 (범위 밖) 등.

이걸 그대로 시뮬에 넘기면 깨진다. 그래서 환경이 들어오는 액션을 자동으로 유효 범위로 보정:

```
theta = float(a[0]) % (2π)          # 0~2π로 wrap
power = clip(a[1], 0, 1)             # 0~1로 clip
ax, ay = clip([a[2], a[3]], -1, 1)
if (ax2 + ay2) > 1:
    ax, ay /= sqrt(ax2 + ay2)      # 단위원 안으로 (miscue 방지)
```

이 보정이 환경 안에 있으니 SAC는 그냥 4개 float 뱉으면 된다.

여기까지 정리: 2단계 끝. 시뮬레이터를 Gym 표준으로 감싸서 SAC가 그대로 학습 가능.

8. 3단계: 환경 변형용 Wrapper

8.1. Wrapper가 뭐야

Wrapper(래퍼) = 기존 환경을 수정 없이 감싸서 동작을 살짝 변형하는 어댑터.

비유: **핸드폰 케이스**. 핸드폰(원본 환경) 자체는 안 바꾸고, 케이스(wrapper)가 외부 동작 — 충격 흡수, 카드 수납 등 — 을 살짝 변형. 핸드폰을 분해하지 않고도 기능 추가.

코드로:

```
env = Billiards4BallInningEnv()      # 원본 환경
env = RandomStartInningEnv(env)      # wrapper로 감쌌 (시작 위치 랜덤화)
env = TimeLimit(env, max_steps=50)   # 또 다른 wrapper (시간 제한)
env = Monitor(env)                   # 또 다른 wrapper (학습 통계)

model = SAC("MlpPolicy", env)        # SAC 입장에선 그냥 환경 하나로 보임
```

여러 wrapper를 양파 껍질처럼 겹쳐 씌울 수 있다.

8.2. Wrapper 패턴이 RL에 좋은 이유

1. 변수 통제 실험 — 환경 코드를 안 건드리고 한 가지만 바꿔서 비교 가능
2. 재사용 — 같은 wrapper를 여러 환경에 끼울 수 있음
3. 조합 가능 — 여러 wrapper 겹쳐 씌우기
4. 알고리즘 무관 — 어떤 RL 알고리즘과도 그대로 호환

8.3. 우리 RL용 wrapper — RandomStartInningEnv

(보상 wrapper 두 개는 RLHF 관련이라 § 16에서 설명한다.)

무엇을 하는가: - inning 환경을 감쌌 - 매 `reset()` 호출 시: 원래 4구 표준 위치 대신 **공 4개를 테이블 안 랜덤한 위치에 다시 배치** - 그 외 step / 보상 / 종료 같은 동작은 원본 그대로

왜 만들었나: 다음 질문 검증. > “정해진 시작 위치에서만 잘 치는 정책이 학습됐을까? 아니면 4구의 일반적 원리를 학습한 걸까?”

만약 정해진 위치에선 잘 치는데 랜덤 위치에선 0%라면 → 정책이 시작 위치에 overfit한 것. 4구의 일반 원리 학습 실패.

구현 디테일: - 쿠션 마진 0.005m (공이 쿠션에 너무 붙어있지 않게) - 공-공 최소 거리 $2R + 0.002\text{m}$ (공끼리 안 겹치게) - Rejection sampling: 랜덤 위치 4개 뽑고 제약 검사, 위반하면 다시 뽑기 - 100번 시도 실패하면 표준 위치로 fallback (학습이 안 멈추게)

이 wrapper가 **Phase I**(나중에 12장에서)의 메인 도구.

여기까지 정리: 3단계 끝. 환경 변형으로 다양한 실험 셋업을 만들 수 있다.

9. 4단계: RL 알고리즘으로 학습 (SAC)

9.1. RL 알고리즘이 하는 일

여기까지 우리가 만든 것: - 1단계: 시뮬레이터 (물리) - 2단계: Gym 환경 (RL이 쓸 수 있는 인터페이스) - 3단계: Wrapper (환경 변형)

이 위에 올라타서 **실제로 학습**을 진행하는 게 RL 알고리즘. 알고리즘은 **정책 함수**(2.2절에서 정의)를 신경망으로 표현하고, 학습 사이클(2.3절)을 돌리며 정책을 점점 좋게 만든다.

알고리즘은 여러 종류가 있다. 우린 메인으로 **SAC** 라는 걸 쓴다.

9.2. SAC가 뭐야 — 처음부터

SAC = Soft Actor-Critic. 2018년 Haarnoja 등이 ICML 학회에서 발표한 알고리즘.

이름 풀이부터:

9.2.1. “Actor-Critic”이 뭐야

RL 알고리즘은 크게 세 학파:

학파 1 — 가치(Value) 기반: > “각 (상태, 행동)이 얼마나 좋은지 점수표(Q-table)를 만들자. 표가 완성되면 매번 점수 높은 행동을 고르면 된다.”

대표: Q-learning, **DQN** (Atari 게임 깬 알고리즘).

문제: 행동이 **불연속(discrete)**일 때만 잘 작동. “왼쪽/오른쪽/점프” 3개 중 고르는 건 OK. 근데 우리 4구는 (θ , power, a , b) — 무한 가짓수의 연속값. 표로 못 만든다.

학파 2 — 정책(Policy) 기반: > “Q-표 만들지 말고, ‘상태 → 행동’ 함수 자체를 신경망으로 만들어서 점수 좋은 방향으로 직접 고치자.”

대표: REINFORCE, **PPO**.

문제: 학습이 **불안정**. 한 번 시도한 결과로만 업데이트하니 운에 휘둘림.

학파 3 — Actor-Critic (둘 다): > “두 개의 신경망을 같이 학습시키자. 하나는 행동 결정(Actor), 하나는 그 결정 평가(Critic).”

신경망	역할	비유
Actor (π)	행동 결정 (정책)	야구 선수 (스윙)
Critic (Q)	그 결정의 가치 평가	코치 (이 스윙은 7점)

작동: 1. Actor가 행동 2. 환경에서 결과(보상) 옴 3. Critic이 그 결과로 “Q값 추정”을 업데이트 (점점 정확해짐) 4. Actor는 Critic 평가 보고 “Q값이 더 높아지는 방향으로” 자기를 업데이트

선수가 스윙하고, 코치가 평가하고, 선수가 평가에 맞춰 스윙을 고친다. 점점 둘 다 좋아진다.

SAC는 학파 3 = Actor-Critic.

9.2.2. “Soft” 부분 — Entropy 보너스

평범한 actor-critic은 **점수만 최대화**한다. 그러면 학습 초반에 운 좋게 1점 낸 행동 하나를 발견하면 → “이거다!” 하고 그것만 무한 반복 → 다른 좋은 행동은 못 찾음 (**local optimum**에 박혀 죽음).

SAC는 점수에 **다양성 보너스(entropy bonus)** 를 더해서 최대화한다:

$$\text{목표} = \mathbb{E}[\text{보상}] + \alpha \cdot H(\pi)$$

- $H(\pi)$: 정책 엔트로피. **정책이 얼마나 다양한 행동을 시도하는가.**
- α : 다양성 가중치 (자동 조정).

엔트로피 의미: - Actor가 하나의 행동만 고집 → 엔트로피 낮음 - Actor가 여러 행동을 비슷한 확률로 섞음 → 엔트로피 높음

비유: 평범 actor-critic = “1번 메뉴가 맛있었네. 평생 1번만 시킬래.” SAC = “1번이 맛있었지만 가끔 2번 3번도 시켜봐야 더 맛있는 게 있을 수도. 고르게 시키자.”

이 양념 덕에 SAC는 **sparse reward**(보상이 가끔만 나오는) 환경에서 강하다. 4구가 정확히 그 영역.

9.2.3. “Off-policy” 양념 — 과거 경험 재활용

SAC는 한 가지 더: **replay buffer** 에 과거 경험을 저장하고 학습할 때 거기서 256개씩 무작위로 꺼내 다시 본다.

비유: 학생이 시험 친 문제집을 한 번만 풀고 버리면 비효율. 같은 문제집을 여러 번 다시 보면서 패턴 발견하는 게 효율적.

이러면 **한 샷 시물을 여러 번 우려먹는다** → 학습 빠름.

9.2.4. SAC 정리 — 세 가지 양념

1. **Off-policy** — 과거 경험 재활용으로 샘플 효율 높음
2. **Continuous action** — actor가 분포로 출력 → 연속 행동 자연스럽게
3. **Entropy bonus** — 다양성 유지, sparse reward에 강함

우리 4구에 잘 맞는 이유: - 연속 action  - Sparse-binary reward → entropy로 탐험 유지  - Off-policy → 시물 재활용 

9.3. PPO 보조

비교용으로 **PPO**(Proximal Policy Optimization)도 같이 돌렸다. PPO는 학파 2(on-policy 정책 기반)의 표준. SAC와 다른 점:

	SAC	PPO
샘플 효율	높음 (재활용)	낮음 (1회용)
안정성	중	높음 (clip trick)
Sparse reward	 강함	약함

우리 결과: SAC 66.7% vs PPO ~33%. **SAC가 우월해서** 메인 알고리즘.

9.4. 우리 코드에서 SAC 사용

`stable-baselines3` 의 SAC를 그대로 사용. 파라미터: - `learning_rate = 3e-4` - `batch_size = 256` - `total_timesteps = 50,000` - 신경망: 256-256 MLP (각 actor / critic)

여기까지 정리: 4단계 끝. 1~3단계 위에 올라타서 SAC가 학습되는 구조.

Part 3 — 부속 인프라와 결과

10. 실험 자동화 인프라

10.1. 한 실험이 너무 느리면 좋은 발견을 못 한다

학습 한 번이 5분이라도 (algo, seed, 하이퍼파라미터) 조합 18개 $\times$ 한 번씩 = **1.5시간**. 이걸 손으로 돌리면 사람이 컴퓨터 옆에 앉아있어야 한다. 그래서 **자동 실행 인프라**를 만들었다.

10.2. experiments/ 디렉터리 구조

```
experiments/
├─ configs.py           # 모든 하이퍼파라미터 한 곳
├─ run_one.py           # 단일 (algo, seed) 실행
├─ run_inning_sac.py    # Phase H 실행
├─ run_inning_matrix.py # Phase H 매트릭스 (algo  $\times$  seed)
├─ run_inning_random.py # Phase I 실행
├─ run_inning_random_matrix.py # Phase I 매트릭스
├─ runs_inning/         # Phase H 결과 저장
├─ runs_inning_random/  # Phase I 결과 저장
└─ results/             # parquet 집계 (분석용)
```

10.3. 디자인 원칙

1. 하이퍼파라미터는 한 곳 (configs.py): 어떤 phase에서도 같은 학습 셋업
2. Seed 3개 표준: 노이즈 측정용 최소
3. Eval은 학습과 분리: 학습 끝난 정책을 deterministic으로 200 episode 평가
4. 결과는 parquet 형식: binary, 압축, 빠르게 읽힘 $\rightarrow$ 분석 노트북이 직접 읽음
5. Random policy baseline 항상 포함: chance-only가 어디인지 명시

11. 분석, 시각화, 페이퍼

11.1. 노트북 — notebooks/

학습 결과를 읽고 figure 만드는 Jupyter notebook들:

노트북	내용
01_env_smoketest	환경이 정상 동작하는지 확인
02_baseline_stats	random/plain SAC 기본 통계
03_inning_demo	inning 환경 동작 데모
06_eval_results	Phase E PPO 평가
09_inning_results	Phase H/I inning 분석

각 노트북에 `.executed.ipynb` 페어가 있어서 결과 캐시 (커밋 가능).

11.2. 렌더 — 브라우저에서 재생

학습된 정책이 어떻게 치는지 눈으로 보고 싶다. `billiards/render/` :

- `replay.py` + `viewer.html` : 한 trajectory 애니메이션 재생
- `dual_replay.py` + `dual_viewer.html` : 두 정책 동시 비교
- 결과물: `artifacts/inning/best_inning.html` 등

11.3. Figure — `paper/figures/`

페이퍼에 들어갈 그래프들 (PDF + PNG 페어): - `fig9_inning_training.pdf` — SAC vs PPO 학습 곡선 - `fig10_inning_distribution.pdf` — inning 길이 분포

(RLHF 관련 figure 1-8은 § 18에서 설명)

12. 지금까지 한 일 (Phase A → I)

프로젝트는 **Phase**(단계)로 진행됐다. 알파벳 순.

Phase A — 환경 / 물리 / 렌더 (✅ 완료)

- 자작 numpy 시뮬 완성 (1단계, 6장)
- `Billiards4BallEnv` Gym wrapper (2단계, 7장)
- HTML viewer
- `tests/` 회귀 테스트
- 검증 figure: `dynamics_check.png` , `collisions_check.png`

Phase B — Inning 환경 (✅)

- `Billiards4BallInningEnv` 추가 (다중샷)
- `tests/test_inning_env.py`

Phase C, D — (RLHF 관련, § 16-20에서 설명)

선호도 라벨러와 보상 모델 학습. 메인 RL solver엔 안 쓴다.

Phase E — PPO Baseline (✅)

- env reward로 PPO 50k step $\times$ 3 seed
- 점수율 ~33% (불안정, seed 편차 큼)
- 노트북 06_eval_results.executed.ipynb

Phase F, G — (RLHF 관련, § 16-20에서 설명)

선형 mix와 PEBBLE 실험. 메인 RL solver엔 안 들어간다.

Phase H — Inning RL (✅)

paper/iclr_inning.md 페이퍼 드래프트 있음.

- SAC + PPO $\times$ 3 seeds $\times$ Billiards4BallInningEnv(max_shots=50)
- 결과:
 - SAC: 66.7% $p \geq 1$, max_inning=1 (한 점 내고 미스)
 - PPO: 33.3% $p \geq 1$
 - random: 0.5%

해석: SAC가 “한 점 내는 정책”은 학습되지만 **multi-shot 정책**(연달아 점수)이 자연스럽게 등장하지 않는다. inning 환경에서도 max=1. 이걸 RL의 한계이지 환경의 한계가 아님 — 정책이 “한 점 내고 미스”에 만족.

→ 이걸 깨는 게 Phase II_e (hierarchical RL) 의 목표 중 하나.

Phase I — Random-Start SAC (🟡 진행 중)

진행 상황: - seed 0만 학습 완료 (runs_inning_random/sac_random_s0/) - canonical eval (정해진 시작 위치): 0% (정시작 위치에서 못 침) - random eval (랜덤 시작 위치): 2.5%, foul 19.5% - **seed 1, 2 미실행**

해석 (예비): - Random-start로 학습한 정책이 정해진 시작 위치에선 0% - → **train/eval distribution**

mismatch: 학습 분포와 평가 분포가 다르면 일반화 실패 - RL의 distribution shift 문제가 그대로 드러남

남은 일: - seed 1, 2 학습 (각 ~7분) - canonical-trained 정책을 random eval에서도 평가 (cross-eval 매트릭스 완성) - paper/iclr_random.md 작성

13. 현재 결과 — Plain SAC 66.7%

13.1. 헤드라인

방법	점수율 (true_score %)	의미
Random policy	~0.5	chance baseline
PPO + env reward	~33	on-policy 기본, 불안정
SAC + env reward	66.7	off-policy + entropy의 위력

3 seed × 50k step × 단발 환경. Eval 200 episode deterministic.

13.2. 왜 SAC가 강한가

(9.2.4절에서 본 그대로) - Off-policy: 샷 재활용으로 효율 - Entropy bonus: sparse reward에서 탐험 유지 - Continuous action: 4-dim 자연스러움

13.3. 한계 — 두 가지 명확

한계 1 — Multi-shot 못 함 (Phase H): - max_inning = 1 - SAC 한 정책이 한 점 내고 그 다음 샷에서 미스 → inning 끝 - 진짜 4구는 5점 10점 누적이 핵심인데 우린 못 함

한계 2 — 시작 위치 일반화 약함 (Phase I 예비): - random start에서 학습 → canonical에서 0% - 정책이 시작 위치에 overfit

Seed 편차도 큼: 3 seed 중 2개가 100% 가까이, 1개가 0% 근처. 기본 SAC는 안정성 부족.

→ Phase II로 이 한계들을 해결하는 게 다음 목표.

Part 4 — 앞으로

14. Phase II 로드맵 — Solver 80%+

목표: Plain SAC 66.7% → 80~85%, multi-shot 정책 발현, 시작 위치 일반화.

여러 개선 후보를 ROI 순으로:

14.1. Phase II_a — 하이퍼파라미터 sweep

가장 가벼운 개선. 단 한 번 학습할 때 쓸 파라미터를 여러 조합 시도.

변수: - learning_rate ∈ {1e-4, 3e-4, 1e-3} - network_size ∈ {256, 512, [256, 256, 256]} - batch_size ∈ {256, 512, 1024} - total_timesteps ∈ {50k, 100k, 200k}

예상 효과: 66.7% → 70~75%. **1-2일 작업.**

14.2. Phase II_b — 다른 알고리즘 비교

SAC를 baseline으로 두고 더 최신 알고리즘 시도.

알고리즘	라이브러리	작업량	기대 효과
TD3 (Twin Delayed DDPG)	SB3 내장	1일	+2~5%
TQC (Truncated Quantile Critics)	SB3-contrib	1일	+5~10%
REDQ (Randomized Ensembled Double Q)	직접 구현	3-4일	샘플 효율 5~10배
CrossQ	직접 구현	3-4일	컴퓨터 절약, 점수 $\approx$

예상 효과: best algo로 +5~10%. **3-5일 작업.**

14.3. Phase II_c — Dense Reward Shaping (사람이 직접 짤)

진짜 점수 보상이 sparse(0/1)이라 학습 초반에 신호 부족. **사람이 직접 룰 기반 dense 보너스를 짜 넣음:**

```

shaped_reward = +1 * score                # 진짜 점수
                + 0.05 * (red ball에 다가감)    # progress
                + 0.02 * (쿠션 1번 친 후 진행)    # 쿠션 컨트롤
                - 0.05 * (파울 위험 - 상대공 근접)  # 파울 방지
                - 0.001 * duration              # 시간 끌면 -

```

중요: - 학습 보상으로만 사용 (wrapper로 적용) - **평가는 진짜 0/1 점수로** (shaped reward는 학습 보조) - 가중치는 사람이 직접 조정 (RM 학습 X)

예상 효과: 66.7% → 80~90%. **2-3일 작업.**

위험: 가중치 잘못 설정하면 어택터 발현 가능 (§ 19에서 자세히). 사람이 통제해야 함.

14.4. Phase II_d (선택) — Curriculum Learning

쉬운 시작 위치 → 점차 표준 위치: 1. 공 2개로 학습 → 3개 → 4개 2. 빨간공이 가까이 → 점점 멀어짐 3. canonical 시작 → random 시작

예상 효과: 학습 안정성, seed 편차 감소. Phase I의 distribution shift 완화.

작업량: 3-4일.

14.5. Phase II_e (선택) — Hierarchical RL

- 높은 레벨: “지금 점수내자 / 다음 수 셋업하자” 결정
- 낮은 레벨: 큐 액션 (θ , power, a, b)

예상 효과: 80%+ 가능, **multi-shot이 자연스럽게 등장** (Phase H의 max_inning=1 깨질 수 있음).

→ 임재환 랩 (SAVO, QMP) 어필 강함.

작업량: 5-7일.

14.6. 4주 시간표 (4/29 시작 가정)

주차	일정	산출물
1주 (4/29-5/5)	Proposal 마감 + Phase II_a (sweep)	proposal.pdf, sweep results
2주 (5/6-5/12)	Phase II_b + Phase I 마무리	algo comparison, Phase I cross-eval
3주 (5/13-5/19)	Phase II_c + (선택) II_d/e	80%+ achieved
4주 (5/20-5/26)	페이퍼 통합 + 발표 + 랩 application	paper, slides

14.7. 백업 플랜

- Phase II_b/c가 80% 못 가면: II_a + 더 긴 학습으로 75%라도. 메시지: “한국 4구 SAC solver”
- Hierarchical 시간 부족: future work으로 넘김
- RLHF 확장 (§ 16-20)은 시간 남으면 — 메인은 RL solver

15. 페이퍼 방향성

15.1. 메인 페이퍼 — “Korean 4-Ball Carom Billiards as an RL Testbed”

구성: 1. Setup: 한국 4구 룰, 자작 시물 (Phase A-B) 2. RL solver: SAC baseline 66.7% (Phase E, H) 3. Generalization: random-start cross-eval (Phase I) 4. Improvements: Phase II_a/b/c → 80%+ 5. Multi-shot capability: Phase II_e (선택) 6. Discussion: 4구를 후속 연구 testbed로 제안

메시지: > “한국 4구라는 한국 특화 sparse-binary 카롬 도메인을 자작 시뮬레이터로 학부생 컴퓨팅 예산에서 학습 가능하게 만들고, SAC 기반 solver를 80%+까지 끌어올렸다. 후속 연구를 위한 깨끗한 testbed로 코드/시물을 공개한다.”

타겟: - CS377 최종 페이퍼 (1차) - 학회 워크샵 — NeurIPS/ICML student abstract, 한국 학회 KSC/KIISE - 이기민/임재환 랩 application 첨부

15.2. 대안 / 확장

Paper B — “Hierarchical RL for Multi-Shot 4-Ball” (Phase II_e 메인)

Multi-shot 발현이 메인 contribution. SAVO/QMP 라인 어필.

Paper C — “Curriculum Learning for Sparse-Binary Carom”

Phase II_d 메인. 점진적 난이도 증가.

Paper D — RLHF 확장 (§ 16-20 참고)

15.3. 어느 페이퍼 메인으로?

추천: Paper A + RLHF 확장 (§ 16-20) 부록.

- Phase II로 충분히 강한 단일 페이퍼 가능
- RLHF는 부록 / 별도 섹션으로 첨부 → 두 랩 모두 어필
- 1개월 안에 가능

Part 5 — RLHF 확장 (별도)

이 Part 5는 메인 프로젝트 (§ 1-15)의 부속. RL solver를 더 끌어올리려는 시도가 어디서 막히고 어디서 풀리는가를 본다. RLHF 자체가 메인이 아니다.

시간 남으면 진행. 메인 우선.

16. RLHF가 뭐야 — 처음부터

16.1. 보상이 명확하지 않을 때

지금까지 우리 RL은 보상이 명확했다: 4구 룰로 0/1. 룰이 명확하니 코드로 짤 수 있다.

근데 어떤 문제는 보상을 코드로 못 짤다: - “좋은 글이란 무엇인가” — 룰로 못 짤 - “이 답변이 도움이 되나” — 사람마다 다름 - “이 그림이 예쁘나” — 주관적

이런 문제에 RL을 쓰려면 보상 함수를 어디서 받아야 할까?

답: 사람한테 받자.

16.2. 사람이 절대 점수는 못 매겨도, 비교는 한다

문제: 사람이 “이 답 8점, 저 답 6점” 같은 절대 점수를 일관되게 못 매긴다. 같은 답을 어제 8점 매겼다가 오늘 6점 매기기도 함.

근데 사람은 두 답 중 어느 게 낫나 비교는 잘 한다. “A가 B보다 낫다”는 일관성이 높다.

이 사실에서 출발한 게 RLHF (Reinforcement Learning from Human Feedback).

16.3. RLHF의 4단계

- [1] 사람이 두 출력(A, B)을 비교 → “A가 B보다 낫다” 라벨
- [2] 비교 라벨들을 모아서 “보상 모델(RM)” 신경망을 학습
→ RM이 어떤 출력에든 점수를 매길 수 있게 됨
- [3] RL 알고리즘(SAC/PP0)이 RM 점수를 보상으로 받아 학습
- [4] 결과: 정책이 사람 선호에 맞게 출력

이게 ChatGPT 만든 방식. GPT는 처음에 인터넷 글로 학습된 다음, RLHF 단계에서 “사람이 좋아하는 답” 으로 정렬됨.

16.4. 핵심 부품 — RM (Reward Model)

RM = (state, action) → 점수 함수를 신경망으로 학습한 것. 비교 라벨에서 학습.

Bradley-Terry 모델 (1952년 통계학에서 빌려온 식):

$$P(\text{A가 B보다 낫다}) = \sigma(r_A - r_B) = \frac{1}{1 + e^{-(r_A - r_B)}}$$

여기서: - r_A, r_B : RM이 A, B에 매긴 점수 - σ : 시그모이드 함수 (0~1로 squash)

해석: - $r_A = r_B \rightarrow$ 확률 0.5 (반반) - $r_A = r_B + 2 \rightarrow$ 확률 ~0.88 (A가 거의 이김)

손실 함수: BCE (Binary Cross-Entropy):

$$\text{loss} = \text{BCE}(\sigma(r_A - r_B), \text{label})$$

라벨은 1.0 (A 이김), 0.0 (B 이김), 0.5 (tie).

이걸 5천 페어 정도로 학습하면 RM이 **상대 순서**(누가 누구보다 낫나)를 학습.

16.5. 우리 4구는 RLHF 적용 가능?

기술적으론 그렇다. 4구 시물에서 두 샷 보여주고 “A가 B보다 낫다”라고 라벨링하면 그 라벨로 RM을 학습할 수 있다.

그러면 정책 **SAC**는 RM 점수를 보상으로 받아 학습.

근데 — 4구는 보상이 명확한 게임인데 굳이 RLHF가 필요할까? 다음 장에서.

17. 4구에 RLHF가 왜 필요한가

17.1. 솔직히 — 4구만 풀려면 RLHF 필요 없음

4구는 점수가 0/1로 명확하다. **Plain SAC 만으로 66.7%**. 굳이 RLHF 안 써도 된다.

그럼 왜 끼우나? 두 가지 동기.

17.2. 동기 1 — 솔직: 랩 어필

이기민(KAIST AI), 임재환(CLVR) 두 교수님 랩이 RLHF/preference learning이 본업. 우리 페이퍼에 RLHF 분석이 들어 있으면 어필 강해진다.

이 동기 자체는 부끄러워할 일 아님. 연구 페이퍼의 80%는 post-hoc rationalization (사후 정당화)다.

17.3. 동기 2 — 연구적: RM이 hand-shaped와 어떻게 다른가

Phase II_c에서 **사람이 직접 짤 dense reward shaping** 으로 80% 도달 (예상). 자연스러운 후속 질문:

“**학습된 RM**(사람이 손으로 안 짜고 라벨로부터 배운 보상 함수) 이 같은 역할을 할 수 있나? 더 잘할 수도 있나?”

만약: - RM > hand-shaped 면 → “라벨링이 손-디자인보다 효율” - RM < hand-shaped 면 → “RLHF가 sparse-binary에서 약함, 왜?”

어느 쪽이든 흥미.

17.4. 4구에서 RLHF로 학습할 만한 것 — 5가지 use case

여기서 흔한 의문: “4구의 어떤 측면에 preference를 줄 거야?” 구체화.

17.4.1. Use Case A — Multi-shot 셋업 품질 (가장 자연스러움) 🌟🌟🌟

문제: Phase H에서 SAC가 max_inning=1. 왜? 정책이 “이 샷 점수내기”만 학습. “점수 내고 + 다음 샷도 좋은 위치” 같은 multi-shot 가치는 학습 안 됨.

진짜 점수로 풀기 어려움: “다음 샷이 좋은 위치”의 정의는 사람마다 다름. 하드코딩 어려움.

Preference로 풀기 좋음: 두 샷 보여주고: - 페어 A: “점수 +1, 그 후 공이 모서리에 박힘 (다음 샷 어려움)” - 페어 B: “점수 +1, 그 후 빨간공 둘이 가까이 모임 (다음 샷 쉬움)” - 사람: “B가 낫다”

이러면 RM이 **shot quality + setup quality** 를 함께 학습. 정책이 inning을 길게 만들 수 있음.

구현: 라벨러가 두 샷의 점수 + 다음 샷 시작 상태(공 분포 entropy)로 채점. RM이 $r(s, a, s')$ 로 next state도 입력에 포함.

평가: max_inning 분포 추적 → max=1 깨면 헤드라인.

17.4.2. Use Case B — 안전 / 파울 회피 강화 🌟🌟

문제: Phase F (§ 18)에서 가끔 정책이 100% 파울로 붕괴. 점수만 강하게 학습하다 위험한 행동.

Preference: 점수 같은 두 샷이라도 “큐볼이 상대공에 0.01m 접근” vs “0.5m 거리 유지” 사이에 안전 선호.

구현: 라벨러에 안전 마진 추가.

왜 의미 있나: 이기민 랩(MobileSafetyBench, RLHF safety) 어필.

17.4.3. Use Case C — 스타일 / 미적 선호 🌟

“예쁜 샷” 라벨링. 객관 평가 어려움 (ground truth 없음). 학부 1개월엔 비추.

17.4.4. Use Case D — 사람 따라하기 쉬운 정책 🌟

너무 복잡한 마세 안 쓰는 정책 학습. 4구 교습 AI 같은 응용. 학부엔 specific.

17.4.5. Use Case E — 라벨러 bias를 연구 대상으로 🌟🌟🌟 (우리 현재 경로)

이게 우리가 실제로 하고 있는 거. Phase F-J 결과의 framing.

문제 정의: > “synthetic teacher의 라벨링 bias가 정책 학습에 어떻게 영향을 주는가?”

연구 가치: - 4구는 ground-truth (진짜 점수)를 알기 때문에 “**hacking이 일어났다**”를 객관적으로 측정 가능 (true_score %) - 다른 RLHF 도메인은 ground-truth 없어 hacking 측정 불가 - 즉 4구는 RLHF reward hacking을 정량 측정할 수 있는 거의 유일한 작은 testbed

→ 두 랩 다 어필.

17.5. 메인 추천 — Use Case E + (시간 허용 시) A

Use Case	1개월 가능성	어필 강도
E. 라벨러 bias 연구	✅ 인프라 있음	매우 강함
A. Multi-shot 셋업	가능 (1주 추가)	강함
B/C/D	(스킵 권장)	중/약

18. 우리가 이미 시도한 RLHF — Phase F, G

§ 17.4.5의 Use Case E 경로로 이미 진행한 실험들. 메인 RL 페이퍼에는 안 들어가지만 **인프라와 결과가 다 존재한다**.

18.1. RLHF용 인프라 (이미 다 만들어져 있음)

보상 wrapper 두 개 — `billiards/wrappers/`

- `RewardModelEnv` (`reward_model_env.py`): env reward(0/1)를 학습된 RM 점수로 완전히 교체. 원본 점수는 `info['env_reward']` 보존.
- `MixedRewardEnv` (`mixed_reward_env.py`): env reward와 RM 점수를 선형 mix:

$$r = \alpha \cdot \text{env_score} + (1 - \alpha) \cdot \text{RM_norm}(s, a)$$

RM 출력은 z-score 정규화 + 클립 [-2, 4]. 정규화 stats는 `experiments/rm_normalization.json` 에 저장 (모든 α sweep이 같은 stats 공유 → 비교 fair).

선호도 데이터 파이프라인 — `billiards/preference/`

- `labeler_heuristic.py` — Synthetic teacher (룰 기반 채점기). 사람 라벨러 흉내내는 모방 함수. 채점 공식 (★ 후술하는 0%의 시드):

$$\text{score}_{\text{labeler}} = +5 \cdot \mathbb{1}_{\text{scored}} - 5 \cdot \mathbb{1}_{\text{foul}} + 0.4 \cdot \min(\text{cushions}, 5) + \dots$$

- 두 샷 점수차 $> 0.5 \rightarrow$ 라벨 'A'/'B' (dead-band). 작음 $\rightarrow$ 'tie'.
- `dataset.py` — `PreferencePair` 직렬화
- 결과: `data/preference_pairs_5k.jsonl` (5천 페어)

보상 모델 (RM) — `billiards/reward_model/`

- `network.py`: `RewardMLP` = 32-dim 입력 $\rightarrow 256 \rightarrow 256 \rightarrow 1$
- `train.py`: Bradley-Terry BCE 손실로 학습
- 학습 완료 모델: `models/reward_model.pt`

PEBBLE outer loop — `billiards/pebble/`

PEBBLE (Lee et al., ICML 2021): SAC + RM **동시 학습** 알고리즘.

루프:

- [1] SAC가 K step 학습 (현재 RM 사용)
- [2] Replay buffer에서 페어 뽑아 라벨링
- [3] RM 업데이트 (누적 데이터)
- [4] Replay buffer 전체를 새 RM으로 relabel
→ [1]로 반복

핵심 트릭 — Relabeling: RM이 변하면 buffer의 옛 reward는 stale. 모든 transition의 reward를 현재 RM으로 다시 계산. 이러면 SAC가 일관된 Q함수 학습.

18.2. Phase F — Linear Mix Sweep 결과

paper/iclr_summary.md 페이퍼 드래프트 있음.

- $\alpha \in \{0.0, 0.1, 0.3, 0.5, 0.7, 1.0\} \times 3 \text{ seeds} \times \text{PPO 50k} = 18 \text{ run}$
- 결과: 모든 α 에서 true_score 0%
- $\alpha = 1.0$ (pure env)도 1/3 seed가 100% foul로 붕괴

18.3. Phase G — PEBBLE 결과

paper/iclr_pebble.md 페이퍼 드래프트 있음.

- sac_env, sac_rm_frozen, pebble_full, pebble_disagree $\times 3 \text{ seeds}$
- 결과:
 - sac_env (plain SAC, RM 안 씬): **66.7%** (메인 페이퍼 baseline)
 - sac_rm_frozen : **0%**
 - pebble_full : **0%**
 - pebble_disagree (active query): **0%**

PEBBLE의 모든 트릭(active query, ensemble, relabeling)을 다 써도 0%.

19. 0% 벽의 정체 — Reward Hacking

이게 우리 RLHF 실험의 핵심 발견. 자세히.

19.1. 먼저 정직하게 — RM 설계가 한몫함

너 의문이 정확함. 0%의 직접 원인은 라벨러 채점 함수에 들어간 쿠션 보너스.

라벨러 채점 공식 다시:

$$\text{score}_{\text{labeler}} = +5 \cdot \mathbb{1}_{\text{scored}} - 5 \cdot \mathbb{1}_{\text{foul}} + 0.4 \cdot \min(\text{cushions}, 5) + \dots$$

여기 굵게 표시한 항이 쿠션 보너스. 한 샷에서 공이 쿠션을 5번 이상 치면 +2.0 보너스.

19.2. 왜 이런 보너스를 넣었나? — 정직한 답

이유 1 — synthetic teacher가 “사람 직관”을 흉내내려고: - 진짜 사람은 점수 0인 두 샷을 비교할 때도 “이게 더 잘 친 거 같아”라고 평가 - 그 직관이 “쿠션 컨트롤 좋다”, “공이 빨간공 쪽으로 갔다” 같은 것 - 라벨러가 이런 직관을 흉내내야 RM이 sparse한 점수만으로는 못 배우는 dense 신호를 얻음

이유 2 — sparse-reward 문제 회피: - 진짜 점수만 라벨에 쓰면 학습 초반 거의 모든 페어가 “둘 다 0점, tie” → RM이 학습할 게 없음 - 보너스로 0점 페어들에도 순서를 매겨야 RM이 의미 있는 신호를 받음

이런 보너스 설계는 PEBBLE / SURF 논문도 함. synthetic teacher 셋업의 표준 컨벤션.

19.3. 그런데 — 보너스 가중치가 너무 컸음

문제는 보너스 자체가 아니라 **강도(0.4)**:

- 점수 보너스: +5
- 쿠션 보너스 (최대): $+0.4 \times 5 = +2.0$
- 즉 쿠션 5번 치면 점수의 **40%에 해당하는 보너스**

이러면 라벨링이: - 페어 A (점수 0, 쿠션 5번): score = 2.0 - 페어 B (점수 0, 쿠션 0번): score = 0.0 - 차이 2.0 → label “A 우세” (dead-band 0.5보다 훨씬 큼)

→ **라벨 데이터 대부분이 “쿠션 많은 게 좋아”** 가 됨. 점수 페어는 sparse, 쿠션 페어는 dense. RM이 그걸 학습함.

19.4. 그래서 정책이 어떻게 망가지는가

라벨러: "쿠션 5번 친 샷 > 쿠션 0번 친 샷" 라벨 다량
 ↓
 RM: 쿠션 횟수에 강하게 양의 가중치 학습
 ↓
 RM이 (state, action) → reward 매기는 함수가 됨:
 $\text{reward}(s, a) \approx (\text{점수 항 작음}) + (\text{쿠션 항 큼})$
 ↓
 SAC가 reward 최대화하려고 → "어떻게든 쿠션 5번 이상 치는 정책" 발견
 ↓
 "쿠션 5번이면 RM 점수 만점, 점수는 0이어도 RM 만점이라 안 올라감"
 ↓
 SAC: "오 이 정책이 RM 점수 가장 높네!" → 어택터 도달

19.5. 분석 증거 (fig3_attribution.pdf)

학습된 정책의 평균 reward를 component별 분해:

항	학습된 정책의 값	해석
점수 보너스 ($+5 \cdot \mathbb{1}_{\text{score}}$)	≈ 0	점수 절대 안 냄
파울 페널티	≈ 0	파울도 안 함
쿠션 보너스	$\approx 2.0 \leftarrow$ 만점	매번 5+ 쿠션
빨간공 접촉	≈ 0	빨간공 거의 안 맞춤
위치 보너스	≈ 0	셋업 안 함

→ 정책이 **쿠션 항만 만족시키는 axis-aligned 어택터에 정확히 도달.**

19.6. 그래서 이걸 “RM 설계 결함”인가, “RLHF의 본질적 문제”인가

둘 다임. 이게 흥미로운 부분.

“설계 결함” 측면: - 보너스 가중치 0.4를 0.04로 줄이면 어택터 약해짐 (검증 예정) - 보너스를 빼고 점수만 라벨에 쓰면 어택터 사라짐 (단 학습 효율 떨어짐)

“본질적 문제” 측면: - 진짜 사람도 같은 bias 있음 — “쿠션 많은 게 멋있어 보임” - ChatGPT RLHF에서도 동일 패턴 — “긴 답변 = 좋다” 라고 사람이 라벨링 → GPT가 무의미하게 길어짐 - synthetic teacher의 “쿠션 보너스 0.4”는 진짜 사람 라벨러의 “긴 답 선호”의 4구 버전

→ 우리 0% 결과는 라벨러를 더 잘 설계하면 일부 회피 가능하지만, 본질적 문제는 사라지지 않음.

19.7. 페이퍼 framing

약한 framing (잘못): > “라벨러를 잘못 짜서 0% 나왔어요”

너무 디펜시브, 발견의 가치 약함.

강한 framing (옳음): > “Synthetic teacher의 어떤 axis에 작은 bias가 있어도(0.4라는 적당한 가중치) 정책은 그 axis를 극대화하는 어택터에 빠진다. 이는 라벨러 설계 결함이 아니라 RM 기반 RL의 본질적 위험이며, 진짜 사람 라벨러에서도 동일 패턴이 보고됨 (e.g., InstructGPT의 답변 길이 bias). 4구는 이 메커니즘을 정량적으로 분리할 수 있는 깨끗한 testbed.”

20. 0%를 깨는 처방 — Phase J, K, L

이미 0%의 원인을 알았으니 처방을 검증한다.

20.1. Phase J — Standard RLHF Remedies

가설: > “ChatGPT가 쓰는 표준 RLHF 처방 (BC prior + KL anchor)을 적용하면 0% 깨진다.”

20.1.1. BC Prior가 뭐야

BC = Behavior Cloning. 전문가 데모(demonstrations)를 supervised 학습으로 따라하기. RL 아님.

데이터: (전문가의 state, 전문가의 action) 페어 1만 개
학습: $\pi(\text{state}) \rightarrow \text{action}$ 을 supervised regression

ChatGPT 만들 때 SFT(Supervised Fine-Tuning) 단계가 정확히 이거. RLHF 시작 전에 사람 데모로 GPT를 한 번 길들임.

우리 4구에 적용: 1. plain SAC를 끝까지 학습 (66.7% 정책) 2. 그 SAC 정책으로 1만 샷 수집 3. 새 신경망을 그 페어로 BC 학습 → “plain SAC 흉내내는 초기 정책” 4. 그 정책을 시작점으로 RM-driven SAC 학습

효과: 시작점이 좋은 정책이라 RM이 약간만 휘게 하지 완전히 망가뜨리지 못함.

20.1.2. KL Anchor가 뭐야

KL = Kullback-Leibler divergence. 두 확률 분포가 얼마나 다른지 재는 거리.

$KL(\pi_{\text{현재}} \parallel \pi_{\text{기준}}) = \text{"지금 정책이 기준 정책에서 얼마나 멀어졌나"}$

RLHF 표준 보상:

$$r_{\text{총합}} = r_{RM} - \beta \cdot KL(\pi \parallel \pi_{\text{기준}})$$

작동: - 기준 정책: plain SAC (또는 BC pretrained) - RM-driven SAC가 “쿠션만 5번 치는 정책”으로 가려 하면 → 기준과 KL 커짐 → 페널티 → “어 너무 멀리 갔네” → 돌아옴

비유: 학생(SAC)을 코치(RM)가 가르치는데, 가끔 코치가 헛소리함. 그래서 옆에 선생님(기준 정책)을 두고 “이상한 답 쓰지 마”라고 잡아줌.

20.1.3. Phase J 실험 매트릭스

Setting	보상	시작 정책
J0_baseline	RM	무작위 (Phase G 재현)
J1_kl_only	$RM - \beta \cdot KL(\pi \parallel \pi_{\text{ref}})$	무작위
J2_bc_only	RM	BC pretrained
J3_bc_kl (메인)	$RM - \beta \cdot KL(\pi \parallel \pi_{\text{BC}})$	BC pretrained

각 $\times 3$ seeds = 12 run.

예상 결과: - J0: 0% (재현) - J1: 20-40% - J2: 30-50% - **J3: 50-70%** (0% 깨는 메인)

20.1.4. 구현 항목

- scripts/train_bc.py — plain SAC rollout 1만 → BC supervised
- KLAnchoredSAC 클래스 — SB3 SAC 상속, actor loss에 KL term 추가
- experiments/run_bc_kl_matrix.py — 매트릭스 실행

작업량: 1주.

20.2. Phase K — Labeler Bias 정량화

가설: > “쿠션 보너스 가중치 w_c 가 어택터 발현의 직접적 임계점이다.”

실험: $w_c \in \{0.0, 0.05, 0.1, 0.2, 0.4, 0.8, 1.6\} \times 3$ seeds = 21 run.

각 w_c 마다: 1. 라벨러 점수 다시 계산 → 새 페어 데이터 2. RM 재학습 3. SAC 학습 (RM only) 4. 점수율 + 평균 쿠션 횟수 측정

예상 figure: - X축: w_c - Y축 1: true_score rate - Y축 2: 평균 쿠션 횟수 - $w_c = 0$: sparse 학습 (점수율 낮음, 쿠션도 낮음) - $w_c \sim 0.1-0.2$: 전이점 (어디서 어택터 발현?) - $w_c = 0.4$ (기본값): 어택터 완전 발현, 0% - $w_c = 1.6$: 100% 쿠션 매닉

Phase J와 병렬 진행 가능.

20.3. (선택) Phase L — SURF Symmetry Augmentation

SURF: Park+ 2022 ICLR 논문. Semi-supervised reward learning with data augmentation.

핵심 트릭 — 대칭성 활용한 데이터 증강:

4구 당구대는 좌우/상하 대칭. 모든 (state, action) 페어를 거울로 뒤집어도 같은 품질의 샷이어야 함.

원래: (state, action_왼쪽으로 침)

거울 뒤집기: (state_거울, action_오른쪽으로 침)

같은 품질 → 같은 RM 점수가 나와야 함. 학습 데이터에 추가: - 5천 페어 → 좌우반전 → 1만 - + 상하반전 → 2만 - + 큐볼 swap → 4만

효과: 라벨 효율 4배. RM이 “오른쪽 위에 있는 빨간공이 좋다” 같은 spurious feature 학습 못 함.

구현: `preference/augment.py` 약 100줄.

20.4. RLHF 페이퍼 메시지

“Synthetic-teacher PbRL이 sparse-binary 보상에서 reward hacking에 빠진다. PEBBLE의 algorithmic 영리함만으론 못 막는다. 표준 처방(BC + KL anchor)이 X% 회복한다. 4구는 이 현상을 정량 검증하기 좋은 testbed.”

중요한 framing: 우리는 **PbRL with synthetic teacher** (가짜 사람 라벨러). 진짜 RLHF (사람 라벨러)는 미래 작업. 페이퍼에 “synthetic” 명시 필수.

20.5. 시간 분배

기본 4주에서 RLHF에 할당: - Phase II로 RL 80% 달성이 우선 - Phase J/K는 시간 남으면 - 시간 충분 → 3주차에 Phase J - 시간 부족 → 부록 처리 - 시간 매우 부족 → future work으로만 언급

부록

21. 코드 / 파일 위치

```

project/
├─ billiards/                # 메인 패키지
│  └─ env.py                 # 단발 Gym (§7)
│  └─ inning_env.py         # inning Gym
│  └─ physics/              # 시뮬레이터 (§6)
│     └─ README.md          # 출처 docs
│     └─ state.py
│     └─ cue_impact.py       # Marlow 큐 임팩트
│     └─ dynamics.py        # 슬립/롤
│     └─ collisions.py       # Han 쿠션 + 공-공
│     └─ simulator.py
├─ render/                  # 렌더 (§11.2)
├─ wrappers/               # Wrapper (§8)
│  └─ random_start_env.py    # RL용
│  └─ reward_model_env.py    # RLHF용 (§18)
│  └─ mixed_reward_env.py    # RLHF용 (§18)
├─ preference/             # RLHF 라벨러 (§18)
├─ reward_model/           # RLHF RM (§18)
├─ pebble/                 # RLHF PEBBLE (§18)
├─ experiments/            # 실험 인프라 (§10)
├─ notebooks/              # 분석 (§11)
├─ paper/                  # 페이퍼 드래프트
│  └─ iclr_summary.md        # Phase F (RLHF)
│  └─ iclr_pebble.md         # Phase G (RLHF)
│  └─ iclr_inning.md         # Phase H (RL)
│  └─ figures/
├─ scripts/                # 학습 스크립트
├─ tests/                  # 회귀 테스트
├─ data/                   # preference_pairs (RLHF)
├─ models/                 # 학습된 RM/policy
├─ policies/
└─ artifacts/              # 시각화 결과

```

22. 용어집

알파벳 순. 등장 순서가 아니라 알파벳이라 각 용어는 해당 섹션을 참고.

용어	풀이 (정의된 섹션)
action	AI가 선택하는 행동. 4구는 (θ , power, a, b) (§ 2.4, § 3.4)
action_space	가능한 action 집합. Gym 약속 (§ 7.2)
actor	행동 결정 신경망. SAC의 정책 π (§ 9.2.1)
agent	RL의 학습 주체. AI (§ 2.1)
BC	Behavior Cloning. 데모 supervised 학습 (§ 20.1.1)
Bradley-Terry	비교 결과로 점수 매기는 모델 (§ 16.4)
canonical layout	4구 표준 시작 위치 (§ 7.3)
carom	포켓 없는 당구. 4구가 카롬 변형 (§ 3.1)
continuous action	행동이 연속값. 4구의 (θ , power, a, b) (§ 3.4)
critic	가치 평가 신경망. SAC의 Q함수 (§ 9.2.1)
curriculum learning	쉬운 과제부터 점진적 학습 (§ 14.4)
dense reward	매 step 신호 있음. Phase II_c의 hand-shaped (§ 14.3)
discrete action	이산 행동 (왼쪽/오른쪽/점프) (§ 9.2.1)
distribution shift	학습 분포와 평가 분포 불일치 (§ 12 Phase I)
entropy bonus	정책 다양성 보너스. SAC의 핵심 (§ 9.2.2)
env reward	환경이 주는 진짜 보상 (§ 16.1)
episode	한 게임 세션 (§ 3.5)
eval	학습 끝난 정책을 deterministic 평가 (§ 10.3)
flatten	다차원 배열을 1D로 (§ 7.2)
foul	4구 룰 위반 (§ 3.2)
Gym / Gymnasium	RL 환경 표준 인터페이스 (§ 7.1)
Han 2005	쿠션 충돌 모델 논문 (§ 6.4.4)
hierarchical RL	높은 레벨 / 낮은 레벨 분리 (§ 14.5)
horizon	episode 길이 (§ 7.5)
inning	한 사람이 점수 내는 동안 계속 치는 회 (§ 3.3)
KL anchor	정책이 기준에서 멀어지면 페널티 (§ 20.1.2)
labeler	페어 라벨러. synthetic = 룰 기반 (§ 18.1)
Marlow 1995	큐 임팩트 모델 책 (§ 6.4.1)
MDP	Markov Decision Process. RL의 수학적 프레임
MLP	Multi-Layer Perceptron. 다층 퍼셉트론 (§ 18.1)
observation / obs	AI가 보는 환경 정보 (§ 2.4)

용어	풀이 (정의된 섹션)
off-policy	과거 경험 재활용. SAC (§ 9.2.3)
on-policy	현재 경험만 사용. PPO (§ 9.3)
PbRL	Preference-based RL. RLHF의 학술 명칭 (§ 16.5)
PEBBLE	SAC + RM 동시 학습 알고리즘 (§ 18.1)
PoolTool	우리 물리 reference 라이브러리 (§ 6.3)
PPO	Proximal Policy Optimization (§ 9.3)
Q함수	상태-행동의 미래 보상 합 추정 (§ 9.2.1)
radial clip	(a, b)를 단위원 안으로 자름 (§ 7.6)
random start	reset마다 공 4개 랜덤 배치 (§ 8.3)
reward	한 행동의 점수 (§ 2.1)
reward hacking	RM 속여서 진짜 목표 안 함 (§ 19)
reward model / RM	비교에서 학습한 점수 함수 (§ 16.4)
reward shaping	dense 보조 보상 (§ 14.3)
RLHF	RL from Human Feedback (§ 16.3)
rollout	한 episode 시물, trajectory
SAC	Soft Actor-Critic. 메인 알고리즘 (§ 9.2)
score	4구 점수, +1 또는 0 (§ 3.2)
seed	랜덤 시드 (§ 10.3)
SFT	Supervised Fine-Tuning. ChatGPT의 BC 단계 (§ 20.1.1)
slip	미끄럼. 공이 굴러가는 첫 단계 (§ 6.4.2)
sparse reward	보상이 드문 환경
SURF	Park+ 2022. 대칭성 augmentation (§ 20.3)
synthetic teacher	룰 기반 라벨러 (사람 대신) (§ 18.1)
TD3	Twin Delayed DDPG. 후보 알고리즘 (§ 14.2)
terminated	episode가 게임 룰로 끝남 (§ 7.2)
trajectory	시물 중 공 위치 시계열
truncated	시간/횟수 제한으로 끊김 (§ 7.2)
wrapper	환경에 덮어 씌우는 어댑터 (§ 8.1)

23. 참고 문헌

23.1. 물리

- Marlow, W. C. (1995). *The Physics of Pocket Billiards*. MAST Publications. ISBN 978-0964537002.
- Han, I. (2005). Dynamics in carom and three cushion billiards. *Journal of Mechanical Science and Technology*, 19(4), 976–984. doi:10.1007/BF02919180
- Kiefl, E. (2024). Pooltool. *JOSS*, 9(101), 7301. doi:10.21105/joss.07301
- Alciatore, D. <https://billiards.colostate.edu/physics/>

23.2. RL 알고리즘

- Haarnoja, T., et al. (2018). Soft actor-critic. *ICML 2018*. [arXiv:1801.01290]
- Schulman, J., et al. (2017). PPO. [arXiv:1707.06347]
- Fujimoto, S., et al. (2018). TD3. *ICML 2018*. [arXiv:1802.09477]
- Kuznetsov, A., et al. (2020). TQC. *ICML 2020*. [arXiv:2005.04269]
- Chen, X., et al. (2021). REDQ. *ICLR 2021*. [arXiv:2101.05982]

23.3. RLHF / PbRL

- Christiano, P. F., et al. (2017). Deep RL from human preferences. *NeurIPS 2017*. [arXiv:1706.03741]
- Lee, K., Smith, L., & Abbeel, P. (2021). PEBBLE. *ICML 2021*. [arXiv:2106.05091]
- Park, J., et al. (2022). SURF. *ICLR 2022*. [arXiv:2203.10050]
- Ouyang, L., et al. (2022). InstructGPT. *NeurIPS 2022*. [arXiv:2203.02155]

23.4. 우리 페이퍼 드래프트

- `paper/iclr_inning.md` (Phase H, RL): “Multi-Shot Reward Closes (or Doesn’t) the Single-Shot Sparsity Gap”
- `paper/iclr_summary.md` (Phase F, RLHF)
- `paper/iclr_pebble.md` (Phase G, RLHF)

24. FAQ

Q1. 왜 layer를 많이 쪼개? A. 변수 통제 실험. 한 변수만 바꿔서 비교가 가능해야 결론이 나옴. layer 분리 안 되어 있으면 한 실험이 여러 변수를 동시에 바꿔서 결론 못 냄.

Q2. PoolTool 안 쓴 게 너무 야심차지 않냐? A. 1주 안에 짤음. PoolTool에서 4개 모듈만 포팅. 출처 명시.

Q3. 4구가 RL solver 잘 돌아가는데 왜 RLHF 끼워? A. § 17 참고. 솔직히 (1) 두 랩 어필 (2) RM-shaped vs hand-shaped 비교가 흥미로움. RL solver 메인 페이퍼는 RLHF 없이도 standalone.

Q4. Phase II에서 80% 못 가면? A. 75%만 가도 OK. 메시지: “한국 4구 SAC solver, Plain 66.7% → improved 75%”. 그대로 작동.

Q5. 컴퓨팅 못 따라오면? A. seed 3 → 2, total_timesteps 50k → 30k 줄여도 헤드라인 유지. Phase II_a/b/c 우선순위 명확.

Q6. 한국 4구가 국제적으로 의미 있어? A. 카롬은 국제(UMB)이고 한국 4구는 그 변형. 영문 RL 논문에 4구는 거의 없으니 novelty 확보. 페이퍼에 “Korean 4-ball is a carom variant played in Korea/East Asia” 부연 필요.

Q7. 0%가 진짜로 깨질까? A. BC + KL이 표준 RLHF 처방이라 학계에서 검증된 거라서 깨질 가능성 높다고 봄. 안 깨지면 그것 자체로 흥미로운 발견.

25. 신규 팀원 가이드

처음 합류했다면 다음 순서로:

1. 이 문서 통독 — 1시간. § 1부터 § 15까지 (RLHF는 § 16부터, 처음엔 스킵 OK)
2. Repo clone + 환경 셋업:

```
git clone <repo>
cd cs377/project
uv sync
uv run pytest tests/ # 회귀 테스트 통과 확인
```

3. 노트북 1개 실행 — notebooks/01_env_smoketest.ipynb 로 환경 정상 동작 확인
4. 한 샷 시뮬해보기:

```
from billiards.env import Billiards4BallEnv
env = Billiards4BallEnv()
obs, _ = env.reset()
action = env.action_space.sample()
obs, reward, term, trunc, info = env.step(action)
print(reward, info['cushion_hits'], info['fouled'])
```

5. paper/iclr_inning.md 읽기 — Phase H의 결과 (RL solver 측면)
6. 현재 진행 phase 잡기:
 - RL 메인 (추천): Phase I seed 1, 2 마무리 또는 Phase II_a sweep
 - RLHF 확장 (시간 남으면): Phase J 코드 셋업
7. 궁금한 거 슬랙에 던지기 — 이 문서나 코드 docstring에 답이 없으면.

문서 최종 수정: 2026-04-29. 다음 업데이트: Phase I 완료 + Phase II_a sweep 결과 (대략 5/12).